

Virtual Memory and Secondary Storage

Week 10 — Associative Lookup, Address Translation, Allocation, and RAID

Dr. Auqib Hamid Lone

Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

September 5, 2026

Where Week 9 Left Us

One hierarchy, one recurring design problem

- ▶ Week 9 placed a small, fast cache between the CPU and a larger, slower main memory.
- ▶ A cache line is found by **mapping** an address and checking a **tag**; a miss fetches a block and may trigger **replacement**.
- ▶ The design goal was not to make all memory fast. It was to make the *common* case fast by exploiting locality.

The question for today

What if the program's address space is larger than physical main memory? Can the same “small fast level stands in for a large slow level” idea work between *DRAM* and *storage*?

Roadmap: Six Questions, One Running Program

1. How can hardware find a value by its **content**, not only by an address? (Associative memory and CAM.)
2. How does a virtual address become a physical address? (Paging and the TLB.)
3. What happens when the needed page is not in main memory? (A page fault and demand paging.)
4. How do operating systems divide main memory among processes? (Partitioning, fragmentation, and allocation.)
5. How do HDDs, optical media, and RAID trade capacity, latency, reliability, and throughput?
6. Can we compress Unit III into a decision procedure before the class test?

Running example

A process has a 16-bit virtual address, 256-byte pages, and 8 physical frames. We will translate virtual address 0x2F34, then follow its page fault and storage consequences.

First: sometimes the lookup key is the content itself.

Motivation: An Address Is Not Always the Best Key

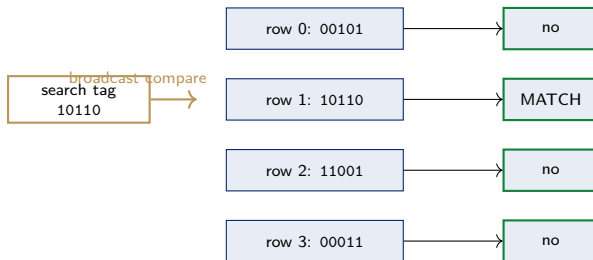
The cache question from Week 9

The CPU presents an address. The cache must answer: “Is the block with this tag present?” Under fully associative mapping, the tag may be in any line, so checking one fixed address is not enough.

- ▶ A conventional RAM lookup is **address** → **contents**.
- ▶ An associative lookup is **content pattern** → **matching row**: compare the search key against many stored entries in parallel.
- ▶ This is powerful but expensive: parallel comparators and match circuitry consume area and energy.

The indexed source’s concrete hardware example is content-addressable memory (CAM), used to make associative cache lookup possible [2, §5.2, PDF pp. 193–194].

CAM: Compare Many Tags at Once



- ▶ The match vector identifies a row; the row's data can then be read.
- ▶ In a TLB, the content being compared is typically a virtual page number plus address-space information; the result supplies a physical frame number.

Socratic Check: What Does Associativity Buy?

Question

A four-entry lookup table receives a key that can match any row. What is the benefit of comparing all four rows in parallel, and what is the cost?

- ▶ **Benefit:** placement is flexible; a key need not be assigned to one predetermined row, reducing conflict misses.
- ▶ **Cost:** four comparisons and match-selection logic operate in parallel; area, wiring, power, and verification complexity increase.
- ▶ **Connection:** fully associative cache mapping and TLB lookup use this same placement-versus-hardware trade-off.

Do not confuse associative memory with “memory that is random.” The adjective describes the lookup method, not the data.

Now use the same hierarchy idea between DRAM and storage.

Motivation: Give Each Process a Private, Large Address Space

The software view

A process should be able to use a simple, contiguous address space without knowing which physical locations are free or which other process owns them.

- ▶ A **virtual address** is generated by the CPU for the process.
- ▶ The memory-management unit (MMU) translates it to a **physical address** in main memory.
- ▶ Pages are the virtual-memory analogue of cache blocks; physical frames are the fixed-size slots that hold them.
- ▶ If the page is absent, the hardware raises a **page fault**; the operating system can fetch it from secondary storage [1, §5.7, p. 441].

The Running System: Virtual Address Fields

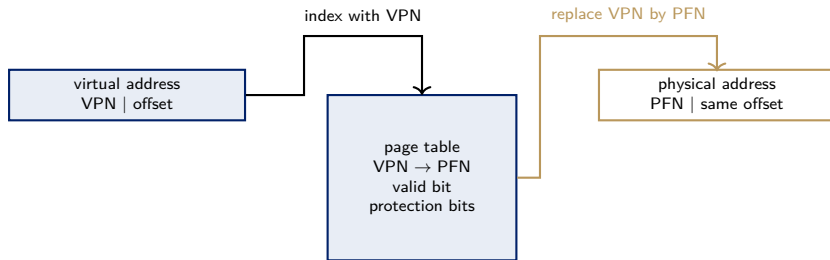
Given

Virtual address width $V = 16$ bits; page size $= 256 = 2^8$ bytes; physical memory has 8 frames $= 2^3$ frames.

- ▶ Virtual address space: 2^{16} bytes = 64 KiB.
- ▶ Number of virtual pages: $2^{16}/2^8 = 2^8 = 256$ pages.
- ▶ Physical memory: $8 \times 256 = 2048$ bytes = 2 KiB.
- ▶ Therefore a virtual address splits into **virtual page number (VPN), 8 bits** and **page offset, 8 bits**.
- ▶ A physical address splits into **physical frame number, 3 bits** and the same **8-bit offset**.

Invariant: translation changes the page/frame field, never the offset within a page.

Paging Translation: The Page Table Is the Map



- ▶ The page table records the current physical frame, validity, and access permissions for each virtual page.
- ▶ A valid entry gives a fast translation; an invalid entry signals that the operating system must intervene.

Worked Example: Translate 0x2F34

Step 1: split the virtual address

0x2F34 = 0010 1111 0011 0100₂. With 8 offset bits:

$$\underbrace{0010\ 1111}_{\text{VPN}=0x2F=47} \quad \underbrace{0011\ 0100}_{\text{offset}=0x34=52} .$$

- ▶ Suppose the page table entry for VPN 47 is valid and maps it to physical frame 5 (101₂).
- ▶ Physical address = $5 \times 256 + 52 = \mathbf{1332}$ bytes.
- ▶ In hex, the physical address is 0x5 || 0x34 = **0x534**.
- ▶ Check: the low eight bits stayed 0x34; only the page number changed.

The TLB: Cache the Translation, Not the Data

Translation Lookaside Buffer

A **TLB** is a small associative cache of recent page-table mappings: VPN $\rightarrow$ PFN. It is checked before the larger page table [1, §5.7, p. 441].

1. CPU emits a virtual address; split it into VPN and offset.
2. TLB compares the VPN against its stored entries in parallel.
3. **TLB hit**: obtain PFN, concatenate PFN with offset, access DRAM.
4. **TLB miss**: consult the page table; if valid, fill the TLB and retry. If invalid, take a page fault.

The TLB is associative because the desired VPN may occupy any TLB entry; its match is a content lookup.

Page Fault: When Translation Finds No Resident Page

A page fault is not automatically a program error

It means the requested virtual page is not currently resident in a physical frame. The operating system must make the reference legal before the instruction can continue.

1. Hardware traps to the operating-system page-fault handler.
2. The handler checks the access: illegal address or permission failure means terminate; otherwise locate the page in storage.
3. If no free frame exists, choose a victim page; write it back if dirty, then read the requested page into the frame.
4. Update the page table and TLB, mark the page valid, and restart the faulting instruction.

This is demand paging: storage is a backing level, not a substitute for the latency of DRAM. The standard hierarchy treatment is covered in Stallings §9.3, PDF pp. 376–389.

Socratic Check: Page Size Is a Trade-off

Question

Why not make every page enormous so that one page fault transfers lots of nearby data? Why not make every page one byte?

- ▶ **Larger pages:** fewer page-table entries and better transfer efficiency when spatial locality is strong, but more wasted space when only a small part of a page is used and more data is moved on a fault.
- ▶ **Smaller pages:** less internal waste and finer protection, but larger page tables and more translation overhead.
- ▶ The useful page size balances page-table cost, transfer cost, fragmentation, and the program's locality.

Paging is not the only way to describe a process's memory.

Segmentation: Preserve Logical Units

A segment is a variable-sized logical region

Instead of splitting an address space into equal pages, segmentation names units such as code, data, stack, or a shared library. A logical address is (**segment number, offset**).

- ▶ A segment table entry supplies a base address, a limit, and access permissions.
- ▶ Translation checks $0 \leq \text{offset} < \text{limit}$, then forms physical address = base + offset.
- ▶ Protection and sharing can follow program structure: code can be read/execute, while data can be read/write.
- ▶ Variable sizes make external fragmentation possible; paging avoids external fragmentation by using fixed-size frames.

Stallings treats segmentation as part of memory management in §9.3, PDF pp. 376–389; the syllabus's Hamacher Ch. 8 is the course-level anchor for the same standard treatment.

Paging and Segmentation: What Problem Are We Solving?

Scheme	Allocation unit	Strength	Main cost
Paging	Fixed-size page/frame	Simple placement; no external fragmentation; easy swapping	Internal fragmentation; page-table/TLB overhead
Segmentation	Variable-size logical segment	Natural protection/sharing; programmer-visible units	External fragmentation; compaction or complex allocation
Paged segmentation	Segments divided into pages	Combines logical protection with fixed-size placement	Two-level translation and more metadata

Exam habit

State the unit first. If the unit is fixed-size, discuss pages, frames, and internal fragmentation. If it is variable-size, discuss segments, holes, and external fragmentation.

Main-Memory Allocation: The Hole-Selection Problem

Contiguous allocation setup

Free memory contains holes of 100 KiB, 500 KiB, 200 KiB, 300 KiB, and 600 KiB. A process requests 212 KiB.

- ▶ **First fit:** place the process in the first hole large enough: 500 KiB. Fast search; leaves a 288 KiB remainder.
- ▶ **Best fit:** choose the smallest adequate hole: 300 KiB. Leaves an 88 KiB remainder, but may create many tiny holes.
- ▶ **Worst fit:** choose the largest hole: 600 KiB. Leaves a 388 KiB remainder, preserving medium-sized holes elsewhere.
- ▶ After placement, the chosen hole is split into an allocated block and a smaller free hole.

Fragmentation: Capacity Can Exist but Be Unusable

Two distinct losses

- ▶ **Internal fragmentation:** unused space inside an allocated fixed-size unit, such as the unused tail of a page or frame.
 - ▶ **External fragmentation:** enough total free space exists, but it is split into non-contiguous holes and cannot satisfy one contiguous request.
-
- ▶ Paging largely removes external fragmentation because any free frame can hold any page.
 - ▶ Paging does not remove internal fragmentation: the last page of a process is often only partly occupied.
 - ▶ Segmentation matches logical units well, but allocation must manage holes; compaction moves allocated segments and is expensive.

Socratic Check: Choose the Allocation Policy

Scenario

A real-time allocator receives many requests between 90 and 120 KiB. It must respond quickly, and future requests are unknown. Which policy is a defensible default, and what evidence would change your choice?

- ▶ **First fit** is a defensible default: it usually searches less and leaves the remainder pattern to be measured rather than guessed.
- ▶ Choose best fit only if measurements show that preserving larger holes is more valuable than the extra search and small-hole production.
- ▶ Choose worst fit only if the workload benefits from retaining useful medium-sized holes and the larger-hole loss is acceptable.
- ▶ The correct engineering answer names the workload and metric, not only the policy label.

A page fault turns a memory reference into a storage request.

HDD Anatomy: Capacity, Position, and Time

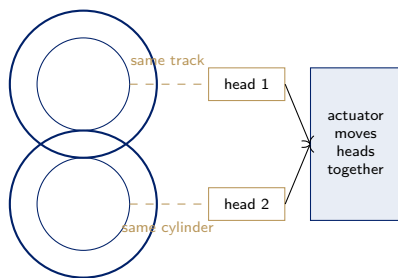
Magnetic disk access is not one constant delay

An HDD stores bits magnetically on rotating platters. A request must select a surface and track, wait for the desired sector to rotate under the head, then transfer the data.

- ▶ **Seek time:** move the actuator to the target track.
- ▶ **Rotational latency:** wait for the sector; average latency is approximately half a rotation.
- ▶ **Transfer time:** read or write the sector once positioned.
- ▶ Approximate access time: $T_{access} = T_{seek} + T_{rotation} + T_{transfer} + T_{controller}$.
- ▶ Sequential access amortizes seek and rotation; random page faults can be dominated by positioning time.

These components and magnetic-disk geometry follow Stallings, §7.1, PDF p. 265.

HDD Geometry: Why Locality Matters Again



- ▶ A cylinder is the set of tracks at the same radius across surfaces.
- ▶ Keeping related blocks nearby reduces movement and improves effective throughput; this is storage-level spatial locality.

RAID: Use Several Disks as One Dependable System

Redundant Array of Independent Disks

RAID combines multiple physical disks and presents a logical storage system. The design dimensions are **striping** for parallelism, **mirroring** or **parity** for recovery, and the number of disks available after redundancy.

Level	Idea	Trade-off
RAID 0	Block striping, no redundancy	High throughput; one disk failure loses the array
RAID 1	Mirroring	Simple recovery; about half raw capacity
RAID 5	Striping with distributed single parity	One-disk failure tolerance; parity-write cost
RAID 6	Striping with distributed dual parity	Two-disk failure tolerance; more parity overhead

RAID levels and their performance/reliability trade-offs are indexed in Stallings §7.2, PDF pp. 276–288, and P&H §5.11, p. 481.

Worked Example: RAID 5 Parity Recovery

Three data blocks and one parity block

Suppose corresponding bytes on three disks are $D_0 = 1010$, $D_1 = 0110$, and $D_2 = 1100$. RAID 5 stores parity $P = D_0 \oplus D_1 \oplus D_2$ on a rotating disk.

- ▶ Compute parity: $1010 \oplus 0110 = 1100$ and $1100 \oplus 1100 = \mathbf{0000}$.
- ▶ If disk D_1 fails, reconstruct it as $D_1 = D_0 \oplus D_2 \oplus P$.
- ▶ $1010 \oplus 1100 \oplus 0000 = \mathbf{0110}$, exactly the missing block.
- ▶ One failure is recoverable because parity preserves the XOR relation; two simultaneous failures require more independent parity, as in RAID 6.

Optical Storage: CD, DVD, and Blu-ray

Read the surface with a laser

Optical media encode information in a spiral track of pits and lands. A laser reads the changing reflectivity; the drive converts that signal into bits.

- ▶ **CD:** compact-disc format; approximately 700 MB for a common data disc.
- ▶ **DVD:** higher-density optical format; approximately 4.7 GB for a single-layer, single-sided disc.
- ▶ **Blu-ray:** shorter-wavelength blue-violet laser and tighter track geometry; commonly 25 GB for a single-layer disc.
- ▶ Read-only, recordable, and rewritable variants differ in how the recording layer changes; optical media are removable and inexpensive per disc, but slower and less convenient for frequent updates than SSDs.

The optical-media treatment is standard secondary-storage content; Stallings indexes CD/DVD/Blu-ray under §7.4, PDF p. 292.

Secondary Storage: Choose by the Workload

Medium	Strength	Weakness	Good fit
HDD	Low cost per bit; large capacity	Mechanical seek and rotation	Bulk data, sequential files, cold storage
SSD/flash	Low latency; no moving head	Write/erase limits; higher cost per bit	OS/apps, random access, active working sets
Optical	Removable; useful for distribution/archive	Slow updates; lower capacity per medium	Media distribution and offline copies
RAID array	Parallelism plus optional redundancy	More controllers, disks, parity or mirror overhead	Servers needing throughput and availability

Design question: Is the bottleneck latency, throughput, capacity, failure recovery, or cost? The medium follows from the bottleneck.

Unit III review: translate, allocate, store, and justify.

One Pattern Across the Memory Hierarchy

The same four-part design pattern

1. **Unit:** line, page, segment, block, or stripe.
 2. **Lookup/placement:** tag comparison, page table/TLB, hole-selection policy, disk geometry, or RAID mapping.
 3. **Miss/failure response:** fetch a block, handle a page fault, split/compact a hole, seek/transfer, or reconstruct from redundancy.
 4. **Cost:** hardware, latency, capacity waste, fragmentation, bandwidth, or redundancy overhead.
- ▶ Week 9: cache makes DRAM appear faster.
 - ▶ Week 10: virtual memory makes storage-backed capacity appear like a larger private memory, while page faults expose the true cost.
 - ▶ No level removes the cost; it changes when and where the cost is paid.

Socratic Check: Diagnose the Failure

Scenario

A program repeatedly touches a small set of pages, but its TLB miss rate and page-fault rate are both high. Which facts should you investigate?

- ▶ **TLB capacity/associativity:** too many active VPNs may evict translations even when all pages are resident.
- ▶ **Physical frames:** too few frames for the working set can cause repeated page faults and thrashing.
- ▶ **Page size/locality:** a poor page size may fetch irrelevant data or create excessive table/translation overhead.
- ▶ **Access pattern:** alternating across many pages can defeat a replacement policy even when total memory seems large.

Unit III Exam Checklist

1. For address translation, split the address into VPN and offset; use the page table/TLB to replace only the VPN; preserve the offset.
2. For a page fault, state the trap, validity/protection check, victim selection, dirty write-back, page-in, table/TLB update, and restart.
3. For paging versus segmentation, name the unit and distinguish internal from external fragmentation.
4. For allocation, show the hole list before and after first fit, best fit, or worst fit; do not merely name the policy.
5. For HDD access, decompose seek, rotational latency, transfer, and controller time; locality reduces positioning cost.
6. For RAID, distinguish striping, mirroring, parity, capacity, throughput, and the number of failures tolerated.
7. For associative memory, explain parallel comparison and its hardware cost; connect CAM/TLB lookup to Week 9's fully associative mapping.

Summary and Bridge to Week 11

Today in one sentence

Virtual memory combines page-based placement, associative translation, and storage-backed recovery so each process can use a protected address space larger than physical DRAM.

- ▶ **Associative memory:** content lookup enables flexible placement; CAM/TLB parallelism costs area and energy.
- ▶ **Paging:** VPN $\rightarrow$ PFN plus unchanged offset; TLB hits are fast, page faults are expensive OS events.
- ▶ **Segmentation/allocation:** logical units and hole policies trade protection, flexibility, and fragmentation.
- ▶ **Storage:** HDD positioning, optical density, SSD latency, and RAID redundancy answer different workload constraints.

Bridge: once memory hierarchy and storage are organized, Week 11 asks how multiple instruction streams and processors can use that system in parallel without creating hazards.

References



David A. Patterson and John L. Hennessy.

Computer Organization and Design: The Hardware/Software Interface.
Morgan Kaufmann, arm edition (5th) edition, 2017.



William Stallings.

Computer Organization and Architecture: Designing for Performance.
Pearson, 10th edition, 2015.

Anchor reading for this week: Stallings 11e Ch. 7 (external memory), Ch. 9 §9.3 (memory management), and Ch. 6 §6.1 (internal memory); Patterson & Hennessy ARM 5e Ch. 5 §5.7 (virtual memory) and §5.11 (RAID); Hamacher Ch. 8 (chapter-level only — not page-verified for this course).